

EXPERIMENT 8

Exploring Jupyter Notebook Advanced Features — Widgets & Interactive Outputs

Aim

To use advanced Jupyter Notebook features — interactive widgets (ipywidgets), rich outputs, and magics — to build interactive data-exploration interfaces.

Theory

Jupyter Notebook is a web application that combines executable code cells, rich text (Markdown), equations (LaTeX) and visualisations in a single document. Each cell runs against a persistent kernel that holds state between executions, enabling exploratory, iterative computing.

ipywidgets adds interactive GUI controls — sliders, dropdowns, buttons, text boxes — that bind to Python callbacks, letting users manipulate parameters and see results update live without editing code. The `@interact` decorator is the quickest way to wire a function to widgets.

Notebooks also support rich display via the `display()` function and `IPython.display` module (HTML, images, audio, DataFrames), and 'magic' commands (`%timeit`, `%matplotlib inline`, `%%writefile`) that extend the interactive workflow.

Software / Tools Required

Jupyter Notebook/Lab, ipywidgets, matplotlib, pandas.

Procedure (Step-by-Step)

Step 1. Install and enable the widget library.

```
pip install ipywidgets matplotlib pandas numpy jupyter
jupyter notebook
```

Step 2. Create an interactive slider bound to a function using `@interact`.

```
from ipywidgets import interact

@interact(n=(1, 20))
def square(n):
    print(f"{n} squared is {n**2}")
```

Step 3. Build a live plot controlled by multiple widgets.

```
import numpy as np, matplotlib.pyplot as plt
from ipywidgets import interact

@interact(freq=(1, 10, 1), amp=(0.5, 5.0, 0.5))
def plot_wave(freq=2, amp=1.0):
    x = np.linspace(0, 2*np.pi, 400)
    plt.plot(x, amp*np.sin(freq*x))
    plt.ylim(-5, 5); plt.show()
```

Step 4. Use explicit widgets and observe their events.

```
import ipywidgets as w
slider = w.IntSlider(min=0, max=100, description="Value")
label = w.Label()
def on_change(change): label.value = f"You chose {change['new']}"
slider.observe(on_change, names="value")
display(slider, label)
```

Step 5. Combine widgets into a layout (VBox/HBox).

```
import ipywidgets as w
dd = w.Dropdown(options=["A","B","C"], description="Class")
btn = w.Button(description="Submit")
display(w.VBox([dd, btn]))
```

Step 6. Render rich outputs (DataFrame, HTML, image).

```
import pandas as pd
from IPython.display import display, HTML
df = pd.DataFrame({"x":[1,2,3], "y":[4,5,6]})
display(df)
display(HTML("<b style='color:teal'>Rich HTML output</b>"))
```

Step 7. Use line and cell magics.

```
%timeit sum(range(1000))
%%writefile hello.py
print("written from a cell")
```

Step 8. Make an interactive DataFrame filter.

```
from ipywidgets import interact
@interact(threshold=(0, 10))
def filt(threshold=5):
    display(df[df.y > threshold])
```

Step 9. Display an inline Matplotlib figure with the magic enabled.

```
%matplotlib inline
df.plot(x="x", y="y", kind="bar")
```

Step 10. Export the notebook to HTML to share interactive results (widgets become static).

```
jupyter nbconvert --to html notebook.ipynb
```

Expected Output

Moving the sliders instantly updates the printed values and the sine-wave/plot outputs without re-running cells manually, and the DataFrame filter responds live — demonstrating interactive, reactive notebook interfaces.

Conclusion

Advanced Jupyter features — ipywidgets, event handling, layouts, rich display and magics — were used to build interactive data-exploration tools inside the notebook.

Viva Questions

1. What is a Jupyter kernel and what state does it hold?
2. What does the @interact decorator do?
3. Differentiate line magics (%) from cell magics (%%).
4. How does the .observe() method wire widget events to callbacks?
5. Why do interactive widgets stop working when a notebook is exported to static HTML?